Marketplace Engine

Technical design document

System architecture, trust boundaries and failure behaviour

Version 1.0 · 13/09/2026

Repository: bradroberts88/marketplace-engine-core

Product codename: AutoPost

Contents

Status: living document. Reflects the code in this repository as of the current release (`autpost-golden` image release, connector `0.1.0`).

1. Purpose and scope

Marketplace Engine (product name **AutoPost**) lets a marketplace operator run Facebook Marketplace sessions for a dealership so that all traffic egresses from **that dealership's own residential/commercial IP address**, not from a datacentre proxy.

This document covers the system as a whole: components, topology, protocols, trust boundaries, failure behaviour, and the hardware provisioning pipeline. Module-level structure is in `SOFTWARE-DESIGN.md`.

Out of scope: the marketplace posting automation itself (separate `production` stack), billing, and CRM integrations.

2. Goals and non-goals

Goals

1. Every rep request for a dealership leaves the internet from that dealership's IP.
2. Fail closed: if the dealership link is not provably alive, the request is refused — never silently rerouted.
3. Zero-touch install at the dealership: plug in a device or run one installer, enter a claim code, done.
4. Never strand a unit in the field: a device on wrong WiFi must be recoverable without a site visit where possible.
5. Fleet-scale operation (target 1,000 dealerships) with signed remote updates and rollback.

Non-goals

- Anonymity or IP rotation. The whole point is a stable, attributable dealership IP.
- General-purpose VPN. The proxy path is scoped to marketplace sessions.
- Cross-platform desktop parity. The desktop connector targets Windows; the appliance path targets Raspberry Pi.

3. System topology

Key property: **the agent always dials out**. Nothing at the dealership needs an inbound port, a static IP, or firewall changes.

4. Components

Component	Location in repo	Runtime	Responsibility
Connector agent	`connector/src/agent.js`	Pi appliance or dealership Windows PC	Maintain the outbound WS control channel, relay streams, heartbeat, apply signed updates
Egress guard	`connector/src/tunnel.js`	With the agent	Open local connections to the target; block private/loopback/link-local/CGNAT ranges (IPv4 + IPv6)
Claim client	`connector/src/claim.js`	First boot / first run	Redeem a one-time setup code, durably write `config.json`
Local dashboard	`connector/src/dashboard.js`	127.0.0.1 on the device	Tunnel health, public IP/geo/latency, rep list
WiFi recovery daemon	`connector/src/wifi-recovery.js`	Pi appliance	Raise an `AutoPost-Setup` AP with a captive page when stranded
Tunnel server	`connector/server/src`	VPS	Control WS, fail-closed CONNECT proxy, claim endpoint, dealership identity store, admin API on localhost
Desktop shell	`connector/electron-main.js`	Windows	Tray app, supervises the agent child process, native dashboard
Keep-alive supervisor	`connector/keep-alive/`	Windows	Scheduled task; restarts a missing or wedged app (heartbeat > 90 s)
Card flasher	`connector/flasher/`	Bench PC	Write and verify the golden image, inject boot files, safety-gate drive selection
Pi deployment set	`connector/deploy/pi/`	Pi appliance	systemd units, install/self-test/verify scripts, USB-gadget SSH, BLE + captive recovery
Golden image kit	`connector/deploy/pi/golden/`	WSL/Linux build host	Build the shipped `.img.xz` (customize-stock-image path)
Build signing	`connector/scripts/sign-build.js`	Release host	Ed25519 detached signatures over update bundles

5. Data plane

1. The rep's browser profile is configured with `custom_proxy = host:port:user:pass`.
2. The proxy authenticates the user and resolves the owning dealership.
3. The server checks that dealership's agent is **LIVE** (WS connected, heartbeat/pong within `heartbeatTimeoutMs`, default 60 s; server pings every 15 s).
4. If live, the byte stream is relayed over the WS control channel; the agent opens the connection locally and pipes bytes back.
5. If not live — never connected, stale heartbeat, or death mid-request — the proxy answers **HTTP 503** and performs no fallback.

The egress guard runs inside the agent, so even a compromised or misconfigured server cannot use an agent to reach the dealership's internal LAN.

6. Control plane

- **Transport:** WSS from agent to server. Agent-initiated only.
- **Authentication:** per-dealership `agentToken` issued at claim time.
- **Liveness:** heartbeat plus server ping/pong; `heartbeat.json` on disk is the local watchdog signal used by the keep-alive supervisor and the self-test.
- **Remote config:** the server may push configuration to the agent; `controlUrl` and the dealership token come from the claim flow and are deliberately not settable from the bench PC.
- **Updates:** update bundles carry an Ed25519 detached signature. The agent verifies against a pinned public key before applying, and rolls back on failed startup. Private signing keys live only on the release host (`$AUTOPOST_SIGNING_KEY`) and are never copied to the VPS.

7. Provisioning and onboarding

1. A VA runs the flasher on the bench PC and selects a removable card. Safety predicates (`flasher/safety.js`) fail closed on anything that is not an eligible removable SD card; a batch requires one explicit confirmation and holds a per-device write lock.
2. Raspberry Pi Imager writes and verifies the golden image; `flasher/inject.js` renders boot partition files: `autopost-claim.env`, `firstrun.sh`, USB-gadget SSH enablement, WiFi profiles (optional — cards can capture WiFi on first boot instead), and a `$6$` console password hash produced by the pure-Node `sha512crypt.js`.
3. On first boot the Pi runs `autopost-claim.service`, redeems the one-time code against the claim endpoint, and writes `config.json` atomically.
4. `autopost-connector.service` starts the agent; `autopost-tailscale.service` provides out-of-band admin access; `autopost-wifi-recovery.service` guards against WiFi stranding.
5. `pi-verify.sh` / `VERIFY-PI.cmd` confirm the unit came up and claimed.

The Windows path is equivalent: `connector/install/install-connector.ps1` registers a hidden `DealershipConnector` scheduled task with restart-on-failure, and the first-run UI redeems the same claim code.

8. Reliability and failure behaviour

Failure	Behaviour
Agent offline	Proxy returns 503. No fallback egress.
Heartbeat stale	Dealership marked down within `heartbeatTimeoutMs`; same 503 path.
App crashes (Windows)	Keep-alive scheduled task relaunches at logon and every minute, on battery too.

Failure	Behaviour
App wedged (Windows)	Heartbeat older than 90 s triggers a forced restart.
Zero-length `config.json`	Guarded by `config-resilience` tests after the 2026-08-30 field failure; writes are atomic.
Mid-session link cut	Pre-cap link refresh gate, guarded by the `planned-refresh` tests.
Wrong WiFi credentials	Device raises `AutoPost-Setup` AP with a captive page; BLE setup path as backup.
Bad update	Signature check rejects unsigned/altered bundles; rollback on failed start.

Only the operator can stop the Windows supervisor, and only via `%LOCALAPPDATA%\AutoPost\disabled.flag`.

9. Security model

Trust boundaries

- Bench PC ↔ card: bench settings hold fleet-wide secrets (Pi console password, Tailscale auth key). They are the same on every shipped unit, so that file must stay off shared drives and out of git. Rotation requires re-flashing; already-shipped cards keep old values.
- Agent ↔ server: mutual over WSS with a per-dealership token; the agent refuses to be pointed at an arbitrary local server.
- Server admin API: bound to localhost only.
- Agent ↔ LAN: the egress guard blocks all private address space, fail closed.

Known accepted risks (tracked in `connector/deploy/pi/PI-SHIP-RISK-REGISTER.md`)

- Fleet-uniform Pi console password and Tailscale key.
- Windows service currently runs as SYSTEM; hardened deployments should substitute a low-privilege service account.
- The golden image is a recipe validated by script syntax checks and unit tests; overlay and power-cut behaviour still needs certification on real hardware.

10. Verification

Pure-Node test suites, no install required (`tools/RUN-TESTS.cmd`):

Suite	Coverage
`flasher/_test/safety`	Never-flash-the-wrong-drive predicates
`flasher/_test/inject`	Golden-file boot artefacts, shell-injection safety
`flasher/_test/nowifi`	Capture-WiFi-on-first-boot cards
`flasher/_test/confirm-batch`, `batch`, `ui-batch`	Batch consent, write locks, lane job ids

Suite	Coverage
`flasher/_test/sha512crypt`	`$6$` hashing parity
`src/_test/planned-refresh`	Pre-cap link refresh gate
`src/_test/config-resilience`	Config write durability + systemd unit presence
`src/_test/wifi-recovery`	Single-radio, never-strand state machine
`server/test-claim-flow.js`	Claim onboarding, health alerts
`connector/test-local.js`	Full loopback: real egress IP + fail-closed proof

Manual gates: `PI-SHIP-TEST-PLAN.md` and `DAY-1-CHECKLIST.md`.

11. Distribution

- Source, scripts and docs: this repository.
- Golden Pi images: GitHub release `autpost-golden` (`autopost-golden.img.xz`, `autopost-golden-zerow.img.xz`), verified by published SHA256.
- Windows installers and packaged Electron output: release assets or the internal file share; build outputs and `node_modules` are deliberately not tracked in git.

12. Open items

- Certify the golden image overlay and power-cut recovery on real Pi hardware.
- Replace the SYSTEM-level Windows task with a scoped service account.
- Per-unit secrets to remove the fleet-uniform password/key exposure.
- Wire the standalone tunnel server into the managed VPS process stack.